

Linked Lists

A linked list is a linear data structure where elements (nodes) are connected via pointers. Unlike arrays, nodes are NOT stored contiguously in memory.

Each node has two parts:

- data - the value
- next - a pointer to the next node (or null)

Here is a linked list with 4 nodes:



The head pointer gives us access to the first node. We traverse by following next until we hit null.

Insert at Head - $O(1)$

BEFORE: head $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ null



We want to insert [5] at the head.

Steps:

1. Create new node [5]
2. Set new node's next to current head

3. Update head to point to new node

AFTER: head $\rightarrow$ [5] $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ null



NO shifting needed! This is why head insertion is $O(1)$.

Delete Operation

To delete a node, we redirect the previous node's next pointer to skip the target.

BEFORE: head $\rightarrow$ [5] $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ null

Delete node [10]:

1. Find the node before [10] (that is [5])
2. Set [5].next = [10].next (which is [20])
3. Free [10]

AFTER: head $\rightarrow$ [5] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ null



Note: Finding the previous node is $O(n)$. But if we already have a pointer to the node before the target, deletion itself is $O(1)$.

comparison: Linked Lists vs Arrays

	Array	Linked List
Access (Index)	$O(1)$	$O(n)$
Search (Search)	$O(n)$	$O(n)$
Insert at front	$O(n)$	$O(1)$
Delete at front	$O(n)$	$O(1)$
Insert at end	$O(1)$	$O(n)$
Delete at end	$O(1)$	$O(n)$

Time Complexity summary

Operation	Time Complexity
Access (Index)	$O(1)$
Search (Search)	$O(n)$
Insert at front	$O(n)$
Delete at front	$O(n)$
Insert at end	$O(1)$
Delete at end	$O(1)$

key tradeoff: Arrays win on random access; linked lists win on dynamic insert/delete at the front.

Linked lists = dynamic size, $O(1)$ insert/delete at head